# Task 1: Weights for the given particles:

| Sigma | Weights | Highest weight index |
|---|---|---|
| 0.05 | [3.40644812e-04, 2.16563930e-02, 1.97339622e-03, 1.59178275e-08, 2.36661324e-01, 7.39063591e-01, 1.41646902e-04, 1.62988550e-04, 6.71746152e-34, 1.81168510e-51] | 5 |
| 0.1 | [5.25591365e-02, 1.48412099e-01, 8.15410594e-02, 4.34553974e-03, 2.69838926e-01, 3.58709789e-01, 4.08803033e-02, 4.37131447e-02, 1.96958203e-09, 7.98166424e-14] | 5 |
| 0.5 | [0.11189864, 0.11664272, 0.1138817, 0.10127929, 0.11946565, 0.12083385, 0.11077951, 0.1110768, 0.05646429, 0.03767755] | 5 |

# Task 2

| num_particles | sigma | delta | Time (s) | Estimate by Particle Filtering |
|---|---|---|---|---|
| 1000 | 0.01 | 0.001 | 1.04 | [ 0.75714294 -0.02582956 -0.71894242] |
| 1000 | 0.01 | 0.01 | 1.03 | [ 0.13441465 0.85971301 -1.25528949] |
| 1000 | 0.01 | 0.1 | 1.04 | [-0.30604849 -0.17822198 0.96037127] |
| 1000 | 0.05 | 0.001 | 1.03 | [ 0.44588286 -0.98929795 2.15166715] |
| 1000 | 0.05 | 0.01 | 1.04 | [ 0.12559531 -0.5507699 -0.78443095] |
| 1000 | 0.05 | 0.1 | 1.07 | [-0.29114586 -0.16662428 1.08796536] |
| 1000 | 0.2 | 0.001 | 1.07 | [-0.00345534 -0.66511368 -0.02110665] |
| 1000 | 0.2 | 0.01 | 1.49 | [-0.00487014 -0.65541879 0.1117227 ] |
| 1000 | 0.2 | 0.1 | 1.22 | [-0.04800871 -0.1383863 0.84049032] |
| 2500 | 0.01 | 0.001 | 2.66 | [ 0.45114565 -0.53867578 -1.95244777] |
| 2500 | 0.01 | 0.01 | 2.68 | [-0.30081384 -0.29817086 0.8994796 ] |
| 2500 | 0.01 | 0.1 | 2.62 | [-0.19561502 -0.14685531 1.28005759] |
| 2500 | 0.05 | 0.001 | 2.59 | [-0.27861996 -0.26508126 0.84675535] |
| 2500 | 0.05 | 0.01 | 2.57 | [-0.30617776 -0.29224814 0.88214467] |
| 2500 | 0.05 | 0.1 | 2.58 | [-0.25896565 -0.17781527 1.17622782] |
| 2500 | 0.2 | 0.001 | 2.58 | [ 0.15178114 -0.53039932 -0.31482695] |
| 2500 | 0.2 | 0.01 | 2.59 | [ 0.19040346 -0.5768816 -0.26957399] |
| 2500 | 0.2 | 0.1 | 2.61 | [-0.05169369 -0.07200875 0.80845656] |

- Increasing the number of particles from 1000 to 2500 led to significantly improved estimation consistency and accuracy. The estimates with 2500 particles reliably converged closer to the ground truth across optimal sigma and delta values. However, this came at the cost of longer runtime, increasing from an average of ~1.05 seconds per trial to ~2.60 seconds per trial.
- Smaller sigma values, such as 0.010, allowed the filter to converge tightly around the true position when an adequate number of particles was used. While larger values (0.200) made the system prone to severe instability and divergence by scattering the particles excessively, the lowest value (0.010) provided the highest precision.
- Delta controls the threshold or step size for updates. A small delta (0.010) led to tighter, highly accurate convergence when using a dense particle cloud. A larger delta (0.100) promoted broader exploration; in cases with fewer particles (1000), increasing delta

actually helped the filter avoid getting stuck in poor local estimates, but with a full 2500 particles, it caused drift and variability in the final pose.

- I use the results from 2500 particles, 0.01 sigma, and 0.01 delta, which has the prediction of [-0.3008, -0.2981, 0.8994].

# Task 3

I estimate the robot base pose with online particle filtering. Each particle is a 2D base hypothesis:

- State: s = [x, y, theta]

Each online observation is a 7-DoF joint vector captured when the stick tip touches an obstacle. Given observation o, I use forward kinematics to get the tip position in the robot frame (x_rel, y_rel), transform it to the world frame for each particle, compute distance d to the closest obstacle boundary, and assign a Gaussian likelihood:

- $w_i \propto \exp(-0.5 * (d_i / \text{sigma})^2)$

After normalization, particles are resampled and perturbed by Gaussian noise (scale delta), with theta wrapped to [-pi, pi]. The final estimate is the particle mean.

## Control and observation

In get_one_obv(sim), the robot actively explores until a touch event is found:

1. Save simulator state.
2. Sample a Cartesian velocity (strong XY, small Z/rotation).
3. Execute one step at a time.
4. Periodically refresh direction.
5. If collision occurs, move briefly in reverse and resample velocity.
6. If touch occurs, record first 7 motor joint angles as observation.
7. Restore simulator state and return observation (or None if timeout).

This provides online measurements while avoiding persistent collision states.

## 3. Pseudocode

### A. get_one_obv

```
function GET_ONE_OBV(sim):
  obv <- None
  saved <- sim.save_state()
  vel <- SAMPLE_VELOCITY()

  for i in 0 .. max_iters-1:
    if i mod refresh_every == 0:
      vel <- SAMPLE_VELOCITY()

    sim.execute(vel)

    if sim.is_touch():
      obv <- first 7 values from sim.get_motor_joint_states()
```

```
            break

        if sim.is_collision():
            repeat retreat_steps times:
                sim.execute(-0.5 * vel)
                if not sim.is_collision(): break
            vel <- SAMPLE_VELOCITY()

    sim.restore_state(saved)
    return obv
```

## B. particle_filter_online

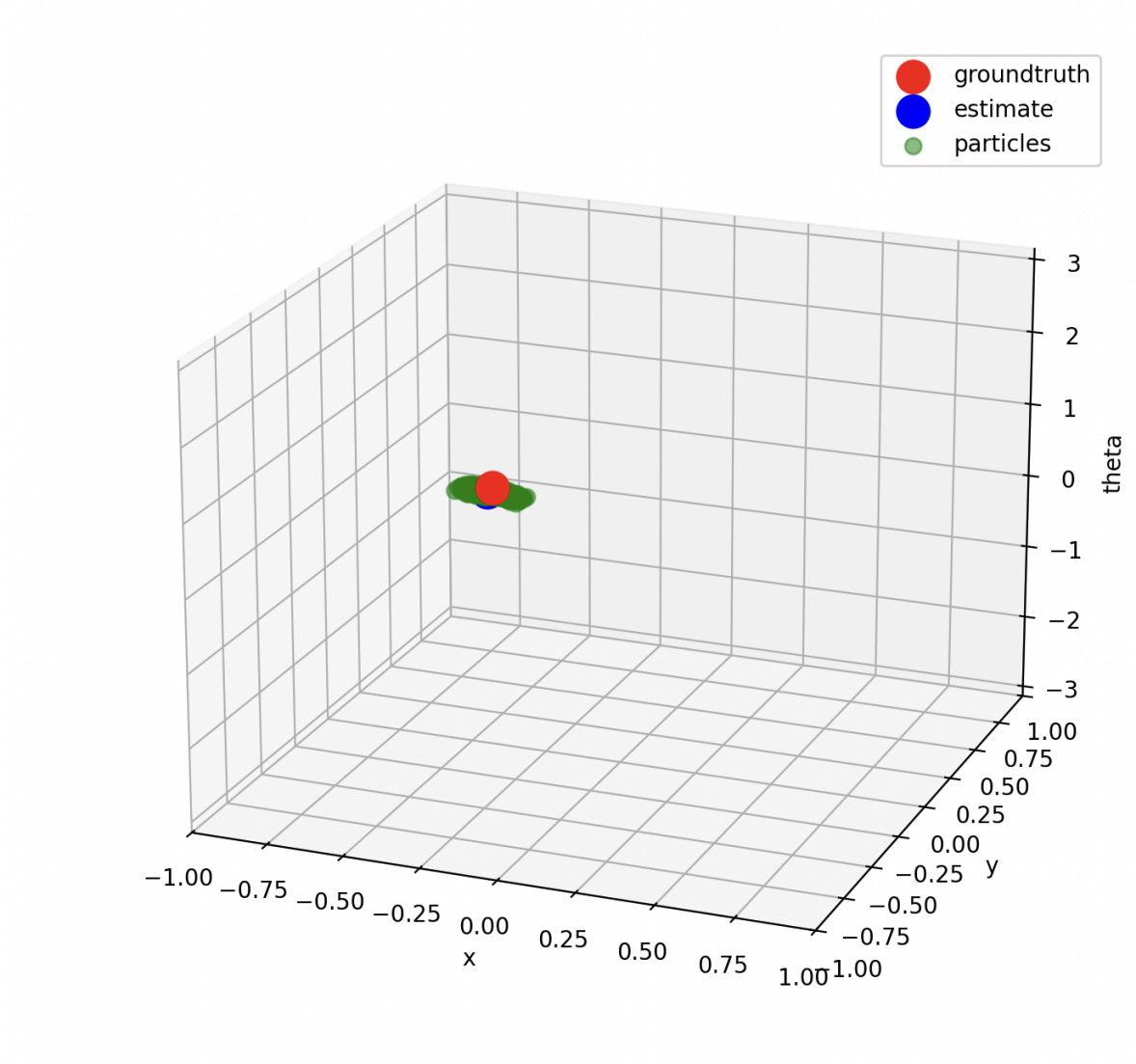
```
function PARTICLE_FILTER_ONLINE(sim, N, sigma, delta):
    particles <- Uniform([x, y, theta], N)
    fail_count <- 0

    for t in 1 .. num_iters:
        obv <- GET_ONE_OBV(sim)

        if obv is None:
            fail_count <- fail_count + 1
            if fail_count >= 2:
                sim.set_joint_values(zeros(7))
                fail_count <- 0
            continue

        fail_count <- 0
        weights <- CAL_WEIGHTS(particles, obv, sigma)
        particles <- RESAMPLE(particles, weights)
        particles <- particles + GaussianNoise(delta)
        particles[:, theta] <- wrap_to_pi(particles[:, theta])

    return mean(particles)
```




I used sigma=0.01, delta=0.01, and num_particles=2500